

WebXR Integration in Virtual Learning Environments

Author: Hang Zhou Student ID: 124100870

1. Introduction

This project explores the integration of WebXR within a Three.js-based virtual learning environment, demonstrating how immersive virtual reality (VR) can be leveraged to enhance interactive education. Unlike traditional 3D-rendered learning platforms, this system provides a fully immersive experience through WebXR, allowing users to interact with digital environments in a natural and intuitive manner using XR controllers, spatial media, and real-time feedback mechanisms.

The core objective is to implement a virtual classroom equipped with multimedia capabilities, interactive objects, and simulated real-world interactions. By integrating WebGL rendering with native `renderer.xr` functionalities, the system extends beyond conventional web-based 3D scenes to a browser-accessible VR-enabled educational platform.

This implementation contributes to ongoing research in WebXR-driven immersive learning, particularly in:

- Embodied cognition and interactive pedagogy – enhancing engagement through natural interaction with digital content.
- Web-based VR accessibility – leveraging WebXR as an open-standard framework to support cross-platform VR experiences.
- Multi-sensory learning experiences – integrating spatial audio, 3D media playback, and real-time lighting manipulation to improve user presence and cognitive retention.

2. WebXR Technical Architecture

2.1 WebXR Renderer Configuration

To enable immersive VR mode, WebXR functionalities are embedded within the WebGL renderer:

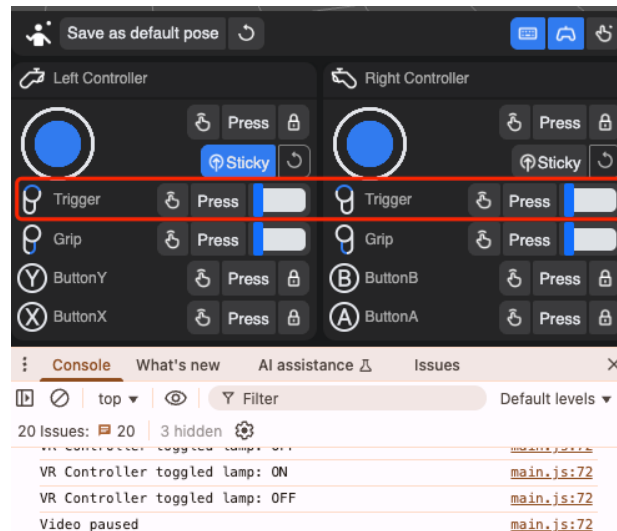
```
renderer.xr.enabled = true;  
document.body.appendChild(VRButton.createButton(renderer));
```

The `VRButton` ensures compliance with modern browser policies on secure gestures and autoplay, allowing users to seamlessly enter an immersive environment.

2.2 XR Controller Interaction System

Dual XR controllers are incorporated to enhance interactive engagement:

```
const controller1 = renderer.xr.getController(0);
const controller2 = renderer.xr.getController(1);
```



Each controller is assigned context-specific interactions within the learning environment:

- Controller 1: Handles synchronized media playback, allowing users to play, pause, and navigate video-based instructional content.
- Controller 2: Toggles dynamic scene lighting, enabling interactive environmental changes such as turning a virtual lamp on/off.

This event-driven architecture enhances real-time interaction, allowing users to manipulate virtual elements using natural hand gestures.

2.3 XR Controller Models and Visualization

To improve spatial awareness and intuitive input handling, controllers are visually represented using `XRControllerModelFactory`:

```
const controllerModelFactory = new XRControllerModelFactory();
controllerGrip1.add(controllerModelFactory.createControllerModel(controllerGrip1));
```

This ensures accurate hand-tracking visualization, reducing cognitive load and improving VR usability.

2.4 Laser Pointer and Raycasting for Object Interaction

A raycasting system is integrated into the controllers, allowing users to interact with objects in the scene using a visualized laser pointer:

```
const geometry = new THREE.BufferGeometry().setFromPoints([...]);
const line = new THREE.Line(geometry, material);
```

```
controller.add(line.clone());
```



This feature supports intuitive UI selection, object manipulation, and interactive scene navigation, paving the way for more complex user interactions such as virtual whiteboards, annotation tools, or interactive simulations.

2.5 Spatial Audio and Sound Implementation

WebXR-based VR environments benefit significantly from spatial audio, which enhances immersion and contextual realism. This project incorporates `THREE.PositionalAudio` to create a responsive sound environment:

```
const listener = new THREE.AudioListener();  
const sound = new THREE.PositionalAudio(listener);  
sound.setRefDistance(1);  
sound.setMaxDistance(10);
```

The sound system enables localized audio feedback, allowing users to experience realistic variations in sound intensity and directionality as they navigate the virtual classroom.

2.6 Dynamic Lighting System

A multi-source lighting model is integrated to enhance the realism of the virtual learning environment:

- Ambient Light: Provides base illumination, ensuring visibility across the entire scene.
- Directional Light: Simulates a projector's illumination, casting realistic shadows and enhancing depth perception.
- Point Light: Configured within a movable lamp structure, which users can toggle via XR controllers.

Each lighting element is dynamically adjustable using GUI controls, allowing real-time modification of intensity, position, and color.

2.7 Shader-Based Visual Effects

To demonstrate GPU-accelerated rendering techniques, the scene includes GLSL shader effects applied to custom geometries (e.g., spheres, torus knots):

- Normal-based shading: Visualizes mesh normals dynamically.
- Distance-based color gradients: Simulates depth perception through radial displacement.
- Sinusoidal UV-based textures: Creates animated wave-like visual effects.

These visual enhancements reinforce the immersive quality of the scene and provide a foundation for more advanced procedural rendering techniques.

2.8 Physics and Collision System

The project implements a robust physics simulation using cannon-es library, featuring:

Ball Physics Implementation:

```
const ballShape = new CANNON.Sphere(ballRadius);
const ballBody = new CANNON.Body({
  mass: 0.5,
  shape: ballShape,
  material: ballPhysicsMaterial,
  position: new CANNON.Vec3(
    Math.random() * 4 - 2,
    ballRadius+0.3,
    Math.random() * 4 - 2
  ),
  linearDamping: 0.01,
  angularDamping: 0.01,
  collisionFilterGroup: BALL_GROUP,
  collisionFilterMask: STATIC_GROUP | CHARACTER_GROUP
});
```

Collision Detection Features:

- Real-time collision detection between the ball and all scene objects
- Physics-based bouncing with adjustable restitution (0.7 by default)
- Custom friction parameters (0.3) for realistic surface interaction
- Collision filtering using bitmask groups (BALL_GROUP, STATIC_GROUP, CHARACTER_GROUP)

Key Components:

- **Physics World Setup**
- **Material System**

- **Collision Bodies:**

- i. All scene objects (tables, chairs, walls, characters) have corresponding physics bodies
- ii. Static objects use mass=0 for optimized simulation
- iii. Dynamic ball object responds realistically to collisions

3. Interaction Design in WebXR

The WebXR framework enables a range of interactive features that contribute to a more engaging learning experience:

3.1 Embodied Interaction

Users can interact with digital objects naturally using XR controllers, reinforcing the connection between physical motion and virtual outcomes.

3.2 Immersion and Presence

The combination of VR headsets, spatial audio, and real-time media interactivity enhances the feeling of “being there,” improving knowledge retention in learning contexts.

3.3 Accessibility and Cross-Platform Deployment

As a browser-based WebXR implementation, this virtual classroom is accessible across multiple devices, including VR headsets, desktops, and mobile devices, ensuring inclusivity for remote learners.

3.4 Physics-Based Interactions

The system demonstrates realistic physical interactions through:

- Ball bouncing with proper energy conservation
- Collision response with all scene objects
- Environment boundaries that prevent object clipping
- Character collision volumes that prevent ball penetration

4. Code Structure and Best Practices

The project follows standard software development practices to ensure maintainability and efficiency:

- **Modular architecture:** Features such as media handling, lighting, and interactivity are encapsulated within separate modules.

- **Commented codebase:** Detailed in-line comments explain key functionalities, facilitating code reuse and extensibility.

- **Performance optimization:** System efficiency is monitored using stats.js, ensuring a smooth VR experience.

- Cross-browser compatibility: The WebXR implementation is tested on major browsers (Chrome, Firefox, Edge) to ensure stability.

5. Conclusion

This WebXR-enabled virtual classroom demonstrates the practical application of immersive VR in educational settings. The integration of realistic physics and collision detection significantly enhances the immersive quality of the virtual learning environment. The ball physics demonstration serves as a foundation for more complex interactive simulations, showcasing the potential for WebXR to deliver engaging, physically-accurate educational experiences. By integrating controller-based interactions, spatial audio, dynamic lighting, and shader-based effects, the system provides a compelling case for the adoption of WebXR in future digital learning platforms. The browser-native implementation ensures broad accessibility, highlighting the potential of WebXR as a transformative tool in interactive education.

6. Asset Attribution

- 3D Models & Animations: Mixamo
- Textures: Poly Haven
- Video Sources: Pexels
- Audio Assets: Pixabay